



数据结构与算法笔记

▼ 目录

- 数据结构类型
 - 数组、链表与哈希表（及列表）
 - 栈
 - 队列
 - 双向队列
 - 二叉树
 - 堆
 - 图
- 搜索
 - 二分查找
 - 二分查找插入点
 - 二分查找（左）边界
 - 哈希优化策略
- 排序
 - 选择排序
 - 冒泡排序
 - 插入排序
 - 归并排序
 - 堆排序
 - 桶排序
 - 计数排序
 - 基数排序
- 分治
 - 分治实现二分查找
 - 构建二叉树
 - 汉诺塔问题
- 回溯
 - 全排列问题
 - 子集和问题
 - n 皇后问题
- 动态规划
 - 动态规划介绍
 - 0-1 背包问题

- 完全背包问题
 - 零钱兑换问题
 - 编辑距离问题
- 贪心
 - 贪心算法介绍
 - 分数背包问题
 - 最大容量问题
 - 最大切分乘积问题

数据结构类型

数组、链表与哈希表（及列表）

特征	数组	链表
存储方式	连续内存空间	分散内存空间
容量扩展	长度不可变	可灵活扩展
内存效率	元素占用内存少	元素占用内存多

对比项	数组	链表	哈希表	说明
查找指定元素	$O(n)$	$O(n)$	$O(1)$	数组、链表需要遍历；哈希表通常为 $O(1)$
指定位置添加元素	$O(n)$	$O(1)$	$O(1)$	数组需要移动元素；链表只需修改指针
指定位置删除元素	$O(n)$	$O(1)$	$O(1)$	数组需要移动元素；链表只需修改指针
添加元素 (需先查找位置)	$O(n)$	$O(n)$	$O(1)$	链表需要先遍历找到位置； 哈希表直接定位
删除元素 (需先查找目标)	$O(n)$	$O(n)$	$O(1)$	链表删除本身快，但查找目标需要遍历
尾部添加元素	$O(1)^*$	$O(1)$	$O(1)$	数组空间足够时；链表维护尾节点时
头部添加元素	$O(n)$	$O(1)$	$O(1)$	数组需要移动元素；链表修改头指针
头部删除元素	$O(n)$	$O(1)$	$O(1)$	数组需要移动元素；链表修改头指针

列表**尾部**添加元素的时间复杂度为 $O(1)$ ，插入和删除元素的效率与数组相同，时间复杂度为 $O(n)$ 。

栈

方法	栈	时间复杂度
push()	元素入栈（添加至栈顶）	$O(1)$
pop()	栈顶元素出栈	$O(1)$
peek()	访问栈顶元素	$O(1)$

栈在基于数组的实现中，入栈和出栈操作都在预先分配好的连续内存中进行，具有很好的缓存本地性，因此效率较高。然而，如果入栈时超出数组容量，会触发扩容机制，导致该次入栈操作的时间复杂度变为 $O(n)$ 。

队列

方法名	队列	时间复杂度
push()	元素入队（将元素添加至队尾）	$O(1)$
pop()	队首元素出队	$O(1)$
peek()	访问队首元素	$O(1)$

双向队列

方法	双向队列	时间复杂度
push_first()	将元素添加至队首	$O(1)$
push_last()	将元素添加至队尾	$O(1)$
pop_first()	删除队首元素	$O(1)$
pop_last()	删除队尾元素	$O(1)$
peek_first()	访问队首元素	$O(1)$
peek_last()	访问队尾元素	$O(1)$

二叉树

方法	无序数组	二叉搜索树
查找元素	$O(n)$	$O(\log n)$
插入元素	$O(1)$	$O(\log n)$
删除元素	$O(n)$	$O(\log n)$

- 层序遍历
 - 时间复杂度为 $O(n)$ ：所有节点被访问一次，使用 $O(n)$ 时间，其中 n 为节点数量。
 - 空间复杂度为 $O(n)$ ：在最差情况下，即满二叉树时，遍历到最底层之前，队列中最多同时存在 $(n + 1)/2$ 个节点，占用 $O(n)$ 空间。
- 前序、中序、后序遍历
 - 时间复杂度为 $O(n)$ ：所有节点被访问一次，使用 $O(n)$ 时间。
 - 空间复杂度为 $O(n)$ ：在最差情况下，即树退化为链表时，递归深度达到 n ，系统占用 $O(n)$ 栈帧空间。

	完美二叉树	链表
第 i 层的节点数量	2^{i-1}	1
高度为 h 的树的叶节点数量	2^h	1
高度为 h 的树的节点总数	$2^{h+1} - 1$	$h + 1$
节点总数为 n 的树的高度	$\log_2(n + 1) - 1$	$n - 1$

在二叉搜索树中不断地插入和删除节点，可能导致二叉树退化为链表，此时各种操作的时间复杂度也会退化为 $O(n)$ 。

堆

方法名	描述	时间复杂度
push()	元素入堆	$O(\log n)$
pop()	堆顶元素出堆	$O(\log n)$
peek()	访问堆顶元素（对于大 / 小顶堆分别为最大 / 小值）	$O(1)$
size()	获取堆的元素数量	$O(1)$

方法名	描述	时间复杂度
isEmpty()	判断堆是否为空	$O(1)$

• 入堆

- 创建一个空堆，然后遍历列表，依次对每个元素执行“入堆操作”，即先将元素添加至堆的尾部，再对该元素执行“从底至顶”堆化。设元素数量为 n ，每个元素的入堆操作使用 $O(\log n)$ 时间，因此该建堆方法的时间复杂度为 $O(n \log n)$ 。

• 遍历堆化

- 将列表所有元素原封不动地添加到堆中，此时堆的性质尚未得到满足。
 - 倒序遍历堆（层序遍历的倒序），依次对每个非叶节点执行“从顶至底堆化”。
- 时间复杂度为 $O(n)$ 。

对各层的“节点数量 × 节点高度”求和，得到所有节点的堆化迭代次数的总和。

$$T(h) = 2^0 h + 2^1 (h-1) + 2^2 (h-2) + \dots + 2^{(h-1)} \times 1 = 2 \frac{1-2^h}{1-2} - h$$

化简得：

$$= 2^{h+1} - h - 2 = O(2^h)$$

进一步，高度为 h 的完美二叉树的节点数量为 $n = 2^{h+1} - 1$ ，易得复杂度为： $O(2^h) = O(n)$

堆入队和出队的时间复杂度均为 $O(\log n)$ ，建堆（入堆）操作为 $O(n)$ 。

• Top-k 问题

Question: 给定一个长度为 n 的无序数组 $nums$ ，返回数组中最大的 k 个元素。

i. 方法一：遍历选择

进行 k 轮遍历，分别在每轮中提取第1、2、...、 k 大的元素，时间复杂度为 $O(nk)$ 。

ii. 方法二：排序

先对数组 $nums$ 进行排序，再返回最右边的 k 个元素，时间复杂度为 $O(n \log n)$ 。

iii. 方法三：堆

- 初始化一个**小顶堆**，其堆顶元素最小。
- 先将数组的前 k 个元素依次入堆。
- 从第 $k+1$ 个元素开始，若当前元素大于堆顶元素，则将堆顶元素出堆，并将当前元素入堆。
- 遍历完成后，堆中保存的就是最大的 k 个元素。

总共执行了 n 轮入堆和出堆，堆的最大长度为 k ，时间复杂度为 $O(n \log k)$ 。当 k 较小时，时间复杂度趋向 $O(n)$ ；当 k 较大时，时间复杂度不会超过 $O(n \log n)$ 。

图

设图中共有 n 个顶点和 m 条边

操作	邻接矩阵	邻接表（链表）	邻接表（哈希表）
判断是否邻接	$O(1)$	$O(n)$	$O(1)$
添加边	$O(1)$	$O(1)$	$O(1)$
删除边	$O(1)$	$O(n)$	$O(1)$
添加顶点	$O(n)$	$O(1)$	$O(1)$
删除顶点	$O(n^2)$	$O(n + m)$	$O(n)$
内存空间占用	$O(n^2)$	$O(n + m)$	$O(n + m)$

图的遍历

• 广度优先遍历

广度优先遍历是一种由近及远的遍历方式，从某个节点出发，始终优先访问距离最近的顶点，并一层层向外扩张，通常借助队列来实现。

算法实现：

- i. 将遍历起始顶点startVet 加入队列，并开启循环。
- ii. 在循环的每轮迭代中，弹出队首顶点并记录访问，然后将该顶点的所有邻接顶点加入到队列尾部。
- iii. 循环步骤2.，直到所有顶点被访问完毕后结束。

◦ 复杂度分析

- 时间复杂度：所有顶点都会入队并出队一次，使用 $O(|V|)$ 时间；在遍历邻接点的过程中，由于是无向图，因此所有边都会被访问2次，使用 $O(2|E|)$ 时间；总体使用 $O(|V| + |E|)$ 时间。
- 空间复杂度：列表 res，哈希集合 visited，队列 que 中的顶点数量最多为 $|V|$ ，使用 $O(|V|)$ 空间。

广度优先遍历的序列不唯一。广度优先遍历只要求按“由近及远”的顺序遍历，而多个相同距离的顶点的遍历顺序允许被任意打乱。

• 深度优先遍历

深度优先遍历是一种优先走到底、无路可走再回头的遍历方式。

算法基于递归实现！

◦ 复杂度分析

- 时间复杂度：所有顶点都会被访问1次，使用 $O(|V|)$ 时间；所有边都会被访问2次，使用 $O(2|E|)$ 时间；总体使用 $O(|V| + |E|)$ 时间。
- 空间复杂度：列表 `res`，哈希集合 `visited` 顶点数量最多为 $|V|$ ，递归深度最大为 $|V|$ ，因此使用 $O(|V|)$ 空间。

深度优先遍历序列的顺序也不唯一。给定某顶点，先往哪个方向探索都可以，即邻接顶点的顺序可以任意打乱。

搜索

1. 二分查找

Question：给定一个长度为 n 的数组 `nums`，元素按从小到大的顺序排列且不重复。请查找并返回元素 $target$ 在该数组中的索引。若数组不包含该元素，则返回-1。

算法：

初始化指针 $i = 0$ 和 $j = n - 1$ ，分别指向数组首元素和尾元素，代表搜索区间 $[0, n - 1]$ 。

接下来循环执行以下步骤：

1. 计算中点索引 $m = \lfloor (i + j) / 2 \rfloor$ 。
2. 判断 `nums[m]` 和 `target` 的大小关系，分为以下三种情况。
3. 当 `nums[m] < target` 时，说明`target`在区间 $[m + 1, j]$ 中，因此执行 $i = m + 1$ 。
4. 当 `nums[m] > target` 时，说明`target`在区间 $[i, m - 1]$ 中，因此执行 $j = m - 1$ 。
5. 当 `nums[m] = target` 时，说明找到`target`，因此返回索引 m 。

若数组不包含目标元素，搜索区间最终会缩小为空。此时返回-1。

- 复杂度分析
 - 时间复杂度为 $O(\log n)$ ：区间每轮缩小一半，因此循环次数为 $\log_2 n$ 。
 - 空间复杂度为 $O(1)$ ：指针 i 和 j 使用常数大小空间。
- 局限性
 - i. 二分查找仅适用于有序数据。
 - ii. 二分查找仅适用于数组。
 - iii. 小数据量下，线性查找性能更佳。

2. 二分查找插入点

1. 无重复元素

Question：给定一个长度为 n 的有序数组 `nums` 和一个元素 `target`，数组不存在重复元素。现将 `target` 插入数组 `nums` 中，并保持其有序性。若数组中已存在元素`target`，则插入到其左方。返回插入后 `target` 在数组中的索引。

算法：

```
def binary_search_insertion_simple(nums: list[int], target: int) -> int:
    """二分查找插入点（无重复元素）"""
    i, j = 0, len(nums) - 1 # 初始化双闭区间 [0, n-1]
    while i <= j:
        m = (i + j) // 2 # 计算中点索引 m
        if nums[m] < target:
            i = m + 1 # target 在区间 [m+1, j] 中
        elif nums[m] > target:
            j = m - 1 # target 在区间 [i, m-1] 中
        else:
            return m # 找到 target，返回插入点 m
    # 未找到 target，返回插入点 i
    return i
```

2. 有重复元素

Question：包含重复元素，其余不变。

算法：

1. 当 `nums[m] < target` 或 `nums[m] > target` 时，说明还没有找到 `target`，因此采用普通二分查找的缩小区间操作，从而使指针 i 和 j 向 `target` 靠近。
2. 当 `nums[m] == target` 时，说明小于 `target` 的元素在区间 $[i, m - 1]$ 中，因此采用 $j = m - 1$ 来缩小区间，从而使指针 j 向小于 `target` 的元素靠近。
循环完成后， i 指向最左边的 `target`， j 指向首个小于 `target` 的元素，因此索引 i 就是插入点。


```
def binary_search_insertion(nums: list[int], target: int) -> int:
    """二分查找插入点（存在重复元素）"""
    i, j = 0, len(nums) - 1 # 初始化双闭区间 [0, n-1]
    while i <= j:
        m = (i + j) // 2 # 计算中点索引 m
        if nums[m] < target:
            i = m + 1 # target 在区间 [m+1, j] 中
        elif nums[m] > target:
            j = m - 1 # target 在区间 [i, m-1] 中
        else:
            j = m - 1 # 首个小于 target 的元素在区间 [i, m-1] 中
    # 返回插入点 i
    return i
```

3. 二分查找（左）边界

Question：给定一个长度为 n 的有序数组 `nums`，其中可能包含重复元素。返回数组中最左一个元素 `target` 的索引。若数组中不包含该元素，则返回-1。

```
def binary_search_left_edge(nums: list[int], target: int) -> int:
    """二分查找最左一个 target"""
    # 等价于查找 target 的插入点
    i = binary_search_insertion(nums, target)
    # 未找到 target，返回 -1
    if i == len(nums) or nums[i] != target:
        return -1
    # 找到 target，返回索引 i
    return i
```

查找右边界

可利用查找最左元素的函数，具体方法为：将查找最右一个 `target` 转化为查找最左一个 `target + 1`。

```
def binary_search_right_edge(nums: list[int], target: int) -> int:
    """二分查找最右一个 target"""
    # 转化为查找最左一个 target + 1
    i = binary_search_insertion(nums, target + 1)
    # j 指向最右一个 target，i 指向首个大于 target 的元素
    j = i - 1
    # 未找到 target，返回 -1
    if j == -1 or nums[j] != target:
        return -1
    # 找到 target，返回索引 j
    return j
```

当数组不包含 target 时，最终*i*和*j*会分别指向首个大于、小于 target 的元素。因此，可以构造一个数组中不存在的元素，用于查找左右边界。

- 查找最左一个 target：可转化为查找 $\text{target} - 0.5$ ，并返回指针*i*。
- 查找最右一个 target：可转化为查找 $\text{target} + 0.5$ ，并返回指针*j*。

4. 哈希优化策略

Question：给定一个整数数组 `nums` 和一个目标元素 `target`，在数组中搜索“和”为 `target` 的两个元素，并返回它们的数组索引。（返回任意一个解即可）

算法：

1. 判断数字 `target - nums[i]` 是否在哈希表中，若是，则直接返回这两个元素的索引。
2. 将键值对 `nums[i]` 和索引 `i` 添加进哈希表。

哈希查找将时间复杂度从两个for循环的 $O(n^2)$ 降至 $O(n)$ 。

```
def two_sum_hash_table(nums: list[int], target: int) -> list[int]:
    """辅助哈希表"""
    # 空间复杂度为 O(n)
    dic = {}
    # 单层循环，时间复杂度为 O(n)
    for i in range(len(nums)):
        if target - nums[i] in dic:
            return [dic[target - nums[i]], i]
        dic[nums[i]] = i
    return []
```

查找算法效率对比

	线性搜索	二分查找	树查找	哈希查找
查找元素	$O(n)$	$O(\log n)$	$O(\log n)$	$O(1)$
插入元素	$O(1)$	$O(n)$	$O(\log n)$	$O(1)$
删除元素	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$
额外空间	$O(1)$	$O(1)$	$O(n)$	$O(n)$
数据预处理	/	排序 $O(n \log n)$	建树 $O(n \log n)$	建哈希表 $O(n)$
数据是否有序	无序	有序	有序	无序

排序

1. 选择排序

算法：

- 1. 初始状态下，所有元素未排序，即未排序（索引）区间为 $[0, n - 1]$ 。
- 2. 选取区间 $[0, n - 1]$ 中的最小元素，将其与索引0处的元素交换。完成后，数组前1个元素已排序。
- 3. 选取区间 $[1, n - 1]$ 中的最小元素，将其与索引1处的元素交换。完成后，数组前2个元素已排序。
- 4. 以此类推。经过 $n - 1$ 轮选择与交换后，数组前 $n - 1$ 个元素已排序。
- 5. 仅剩的一个元素必定是最大元素，无须排序，因此数组排序完成。

```
def selection_sort(nums: list[int]):  
    """选择排序"""  
    n = len(nums)  
    # 外循环：未排序区间为 [i, n-1]  
    for i in range(n - 1):  
        # 内循环：找到未排序区间内的最小元素  
        k = i  
        for j in range(i + 1, n):  
            if nums[j] < nums[k]:  
                k = j # 记录最小元素的索引  
        # 将该最小元素与未排序区间的首个元素交换  
        nums[i], nums[k] = nums[k], nums[i]
```

- 算法特性

- 时间复杂度为 $O(n^2)$ 、非自适应排序：外循环共 $n - 1$ 轮，第一轮未排序区间长度为 n ，最后一轮未排序区间长度为2，求和为 $\frac{(n-1)(n+2)}{2}$ 。
- 空间复杂度为 $O(1)$ 、原地排序：指针 i 和 j 使用常数大小的额外空间。
- 非稳定排序：元素 `nums[i]` 有可能被交换至与其相等的元素的右边，导致两者的相对顺序发生改变。

2. 冒泡排序

算法：设数组的长度为 n

1. 首先，对 n 个元素执行“冒泡”，将数组的最大元素交换至正确位置。
2. 接下来，对剩余 $n - 1$ 个元素执行“冒泡”，将第二大元素交换至正确位置。
3. 以此类推，经过 $n - 1$ 轮“冒泡”后，前 $n - 1$ 大的元素都被交换至正确位置。
4. 仅剩的一个元素必定是最小元素，无须排序，因此数组排序完成。

某轮“冒泡”中没有执行任何交换操作，说明数组已经完成排序，可直接返回结果。可以增加一个标志位 `flag` 来监测。

```
def bubble_sort_with_flag(nums: list[int]):  
    """冒泡排序（标志优化）"""  
    n = len(nums)  
    # 外循环：未排序区间为 [0, i]  
    for i in range(n - 1, 0, -1):  
        flag = False # 初始化标志位  
        # 内循环：将未排序区间 [0, i] 中的最大元素交换至该区间的右端  
        for j in range(i):  
            if nums[j] > nums[j + 1]:  
                # 交换 nums[j] 与 nums[j + 1]  
                nums[j], nums[j + 1] = nums[j + 1], nums[j]  
                flag = True # 记录交换元素  
        if not flag:  
            break # 此轮“冒泡”未交换任何元素，直接跳出
```

- 算法特性

- 时间复杂度为 $O(n^2)$ 、自适应排序：各轮“冒泡”遍历的数组长度依次为 $n - 1$ 、 $\dots$ 、2、1，总和为 $(n - 1)n/2$ 。在引入 `flag` 优化后，最佳时间复杂度可达到 $O(n)$ 。
- 空间复杂度为 $O(1)$ 、原地排序：指针 i 和 j 使用常数大小的额外空间。
- 稳定排序：由于在“冒泡”中遇到相等元素不交换。

3. 插入排序

算法：在未排序区间选择一个基准元素，将该元素与其左侧已排序区间的元素逐一比较大小，并将该元素插入到正确的位置。

1. 初始状态下，数组的第1个元素已完成排序。
2. 选取数组的第2个元素作为 `base`，将其插入到正确位置后，数组的前2个元素已排序。
3. 选取第3个元素作为 `base`，将其插入到正确位置后，数组的前3个元素已排序。
4. 以此类推，在最后一轮中，选取最后一个元素作为 `base`，将其插入到正确位置后，所有元素均已排序。

```
def insertion_sort(nums: list[int]):  
    """插入排序"""  
    # 外循环：已排序区间为 [0, i-1]  
    for i in range(1, len(nums)):  
        base = nums[i]  
        j = i - 1  
        # 内循环：将 base 插入到已排序区间 [0, i-1] 中的正确位置  
        while j >= 0 and nums[j] > base:  
            nums[j + 1] = nums[j] # 将 nums[j] 向右移动一位  
            j -= 1
```

• 算法特性

- **时间复杂度为 $O(n^2)$ 、自适应排序：**在最差情况下，每次插入操作分别需要循环 $n - 1$ 、 $n - 2$ 、...、2、1次，求和得到 $(n - 1)n/2$ ，因此时间复杂度为 $O(n^2)$ 。在遇到有序数据时，插入操作会提前终止。当输入数组完全有序时，插入排序达到最佳时间复杂度 $O(n)$ 。
- **空间复杂度为 $O(1)$ 、原地排序：**指针 i 和 j 使用常数大小的额外空间。
- **稳定排序：**在插入操作过程中，我们会将元素插入到相等元素的右侧，不会改变它们的顺序。

4. 插入排序

算法：快速排序的核心操作是“哨兵划分”，其目标是：选择数组中的某个元素作为“基准数”，将所有小于基准数的元素移到其左侧，而大于基准数的元素移到其右侧。哨兵划分完成后，原数组被划分成三部分：左子数组、基准数、右子数组，且满足“左子数组任意元素 $\leq$ 基准数 $\leq$ 右子数组任意元素”。

• 哨兵划分：

- i. 选取数组最左端元素作为基准数，初始化两个指针 i 和 j 分别指向数组的两端。
- ii. 设置一个循环，在每轮中使用 i (j) 分别寻找第一个比基准数大 (小) 的元素，然后交换这两个元素。

iii. 循环执行步骤ii.，直到 i 和 j 相遇时停止，最后将基准数交换至两个子数组的分界线。

1. 首先，对原数组执行一次“哨兵划分”，得到未排序的左子数组和右子数组。
2. 然后，对左子数组和右子数组分别递归执行“哨兵划分”。
3. 持续递归，直至子数组长度为 1 时终止，从而完成整个数组的排序。

```
class QuickSort:
    """快速排序类"""
    def partition(self, nums: list[int], left: int, right: int) -> int:
        """哨兵划分"""
        # 以 nums[left] 为基准数
        i, j = left, right
        while i < j:
            while i < j and nums[j] >= nums[left]:
                j -= 1 # 从右向左找首个小于基准数的元素
            while i < j and nums[i] <= nums[left]:
                i += 1 # 从左向右找首个大于基准数的元素
            # 元素交换
            nums[i], nums[j] = nums[j], nums[i]
        # 将基准数交换至两子数组的分界线
        nums[i], nums[left] = nums[left], nums[i]
        return i # 返回基准数的索引

    def quick_sort(self, nums: list[int], left: int, right: int):
        """快速排序"""
        # 子数组长度为 1 时终止递归
        if left >= right:
            return
        # 哨兵划分
        pivot = self.partition(nums, left, right)
        # 递归左子数组、右子数组
        self.quick_sort(nums, left, pivot - 1)
        self.quick_sort(nums, pivot + 1, right)
```

• 算法特性

- **时间复杂度为 $O(n \log n)$ 、非自适应排序**：在平均情况下，哨兵划分的递归层数为 $\log n$ ，每层中的总循环数为 n ，总体使用 $O(n \log n)$ 时间。在最差情况下，每轮哨兵划分操作都将长度为 n 的数组划分为长度为 0 和 $n - 1$ 的两个子数组，此时递归层数达到 n ，每层中的循环数为 n ，总体使用 $O(n^2)$ 时间。
- **空间复杂度为 $O(n)$ 、原地排序**：在输入数组完全倒序的情况下，达到最差递归深度 n ，使用 $O(n)$ 栈帧空间。排序操作是在原数组上进行的，未借助额外数组。

- **非稳定排序**：在哨兵划分的最后一步，基准数可能会被交换至相等元素的右侧。

可以在数组中选取三个候选元素（通常为数组的首、尾、中点元素），并将这三个候选元素的中位数作为基准数。这样基准数“既不太小也不太大”的概率将大幅提升。采用此法后，时间复杂度劣化至 $O(n^2)$ 的概率大大降低。

5. 归并排序

算法：

- “划分阶段”从顶至底递归地将数组从中点切分为两个子数组。
 - i. 计算数组中点 `mid`，递归划分左子数组（区间 `[left, mid]`）和右子数组（区间 `[mid + 1, right]`）。
 - ii. 递归执行步骤i，直至子数组区间长度为 1 时终止。
- “合并阶段”从底至顶地将左子数组和右子数组合并为一个有序数组。

归并排序与二叉树后序遍历的递归顺序是一致的。

```

def merge(nums: list[int], left: int, mid: int, right: int):
    """合并左子数组和右子数组"""
    # 左子数组区间为 [left, mid], 右子数组区间为 [mid+1, right]
    # 创建一个临时数组 tmp , 用于存放合并后的结果
    tmp = [0] * (right - left + 1)
    # 初始化左子数组和右子数组的起始索引
    i, j, k = left, mid + 1, 0
    # 当左右子数组都还有元素时, 进行比较并将较小的元素复制到临时数组中
    while i <= mid and j <= right:
        if nums[i] <= nums[j]:
            tmp[k] = nums[i]
            i += 1
        else:
            tmp[k] = nums[j]
            j += 1
        k += 1
    # 将左子数组和右子数组的剩余元素复制到临时数组中
    while i <= mid:
        tmp[k] = nums[i]
        i += 1
        k += 1
    while j <= right:
        tmp[k] = nums[j]
        j += 1
        k += 1
    # 将临时数组 tmp 中的元素复制回原数组 nums 的对应区间
    for k in range(0, len(tmp)):
        nums[left + k] = tmp[k]

```

```

def merge_sort(nums: list[int], left: int, right: int):
    """归并排序"""
    # 终止条件
    if left >= right:
        return # 当子数组长度为 1 时终止递归
    # 划分阶段
    mid = (left + right) // 2 # 计算中点
    merge_sort(nums, left, mid) # 递归左子数组
    merge_sort(nums, mid + 1, right) # 递归右子数组
    # 合并阶段
    merge(nums, left, mid, right)

```


- **算法特性**

- **时间复杂度为 $O(n \log n)$ 、非自适应排序**：划分产生高度为 $\log n$ 的递归树，每层合并的总操作数量为 n ，因此总体时间复杂度为 $O(n \log n)$ 。
- **空间复杂度为 $O(n)$ 、非原地排序**：递归深度为 $\log n$ ，使用 $O(\log n)$ 大小的栈帧空间。合并操作需要借助辅助数组实现，使用 $O(n)$ 大小的额外空间。
- **稳定排序**：在合并过程中，相等元素的次序保持不变。

对于链表，归并排序相较其他排序算法具有显著优势，可以将链表排序任务的空间复杂度优化至 $O(1)$ 。

6. 堆排序

算法： 设数组的长度为 n ， 输入数组并建立小顶堆，此时最小元素位于堆顶。2. 不断执行出堆操作，依次记录出堆元素，即可得到从小到大排序的序列。

1. 输入数组并建立大顶堆。完成后，最大元素位于堆顶。
2. 将堆顶元素（第一个元素）与堆底元素（最后一个元素）交换。完成交换后，堆的长度减 1，已排序元素数量加 1。
3. 从堆顶元素开始，从顶到底执行堆化操作（sift down）。完成堆化后，堆的性质得到修复。
4. 循环执行第 2 步和第 3 步。循环 $n - 1$ 轮后，即可完成数组排序。

```
def sift_down(nums: list[int], n: int, i: int):
    """堆的长度为 n，从节点 i 开始，从顶至底堆化"""
    while True:
        # 判断节点 i, l, r 中值最大的节点，记为 ma
        l = 2 * i + 1
        r = 2 * i + 2
        ma = i
        if l < n and nums[l] > nums[ma]:
            ma = l
        if r < n and nums[r] > nums[ma]:
            ma = r
        # 若节点 i 最大或索引 l, r 越界，则无须继续堆化，跳出
        if ma == i:
            break
        # 交换两节点
        nums[i], nums[ma] = nums[ma], nums[i]
        # 循环向下堆化
        i = ma
```

```
def heap_sort(nums: list[int]):
    """堆排序"""
    # 建堆操作：堆化除叶节点以外的其他所有节点
    for i in range(len(nums) // 2 - 1, -1, -1):
        sift_down(nums, len(nums), i)
    # 从堆中提取最大元素，循环 n-1 轮
    for i in range(len(nums) - 1, 0, -1):
        # 交换根节点与最右叶节点（交换首元素与尾元素）
        nums[0], nums[i] = nums[i], nums[0]
        # 以根节点为起点，从顶至底进行堆化
        sift_down(nums, i, 0)
```

- **算法特性**
- **时间复杂度为 $O(n \log n)$ 、非自适应排序**：建堆操作使用 $O(n)$ 时间。从堆中提取最大元素的时间复杂度为 $O(\log n)$ ，共循环 $n - 1$ 轮。
- **空间复杂度为 $O(1)$ 、原地排序**：几个指针变量使用 $O(1)$ 空间。元素交换和堆化操作都是在原数组上进行的。
- **非稳定排序**：在交换堆顶元素和堆底元素时，相等元素的相对位置可能发生变化。

7. 桶排序

算法： 考虑一个长度为 n 的数组，其元素是范围 $[0, 1)$ 内的浮点数。

1. 初始化 k 个桶，将 n 个元素分配到 k 个桶中。
2. 对每个桶分别执行排序（这里采用编程语言的内置排序函数）。
3. 按照桶从小到大的顺序合并结果。

```
def bucket_sort(nums: list[float]):  
    """桶排序"""  
    # 初始化 k = n/2 个桶，预期向每个桶分配 2 个元素  
    k = len(nums) // 2  
    buckets = [[] for _ in range(k)]  
    # 1. 将数组元素分配到各个桶中  
    for num in nums:  
        # 输入数据范围为 [0, 1)，使用 num * k 映射到索引范围 [0, k-1]  
        i = int(num * k)  
        # 将 num 添加进桶 i  
        buckets[i].append(num)  
    # 2. 对各个桶执行排序  
    for bucket in buckets:  
        # 使用内置排序函数，也可以替换成其他排序算法  
        bucket.sort()  
    # 3. 遍历桶合并结果  
    i = 0  
    for bucket in buckets:  
        for num in bucket:  
            nums[i] = num  
            i += 1
```

- **算法特性**

- **时间复杂度为 $O(n + k)$** ：假设元素在各个桶内平均分布，那么每个桶内的元素数量为 $\frac{n}{k}$ 。假设排序单个桶使用 $O(\frac{n}{k} \log \frac{n}{k})$ 时间，则排序所有桶使用 $O(n \log \frac{n}{k})$ 时间。当桶数量 k 比较大时，时间复杂

度则趋向于 $O(n)$ 。合并结果时需要遍历所有桶和元素，花费 $O(n + k)$ 时间。最差情况下，所有数据被分配到一个桶中，且排序该桶使用 $O(n^2)$ 时间。

- **空间复杂度为 $O(n + k)$ 、非原地排序**：需要借助 k 个桶和总共 n 个元素的额外空间。
- 桶排序是否稳定取决于排序桶内元素的算法是否稳定。

桶排序适用于处理体量很大的数据。

8. 计数排序

算法：通过统计元素数量来实现排序，通常应用于整数数组。最大元素记为 m 。

1. 遍历数组 `nums`，统计每个元素出现的次数，存入计数数组 `counter`。
2. 将 `counter` 转换为前缀和，使 `counter[num]` 表示小于等于 `num` 的元素数量。通过前缀和确定每个元素在排序数组中的最终位置。
3. 倒序遍历原数组 `nums`：根据 `counter[num]` 找到元素 `num` 的存放位置。将 `num` 放入结果数组 `res`。更新 `counter[num]`，为下一个相同元素腾出位置。
4. 将排序完成的 `res` 覆盖到 `nums`。

```
def counting_sort(nums: list[int]):  
    """计数排序"""  
    # 完整实现，可排序对象，并且是稳定排序  
    # 1. 统计数组最大元素 m  
    m = max(nums)  
    # 2. 统计各数字的出现次数  
    # counter[num] 代表 num 的出现次数  
    counter = [0] * (m + 1)  
    for num in nums:  
        counter[num] += 1  
    # 3. 求 counter 的前缀和，将“出现次数”转换为“尾索引”  
    # 即 counter[num]-1 是 num 在 res 中最后一次出现的索引  
    for i in range(m):  
        counter[i + 1] += counter[i]  
    # 4. 倒序遍历 nums，将各元素填入结果数组 res  
    # 初始化数组 res 用于记录结果  
    n = len(nums)  
    res = [0] * n  
    for i in range(n - 1, -1, -1):  
        num = nums[i]  
        res[counter[num] - 1] = num # 将 num 放置到对应索引处  
        counter[num] -= 1 # 令前缀和自减 1，得到下次放置 num 的索引  
    # 使用结果数组 res 覆盖原数组 nums  
    for i in range(n):  
        nums[i] = res[i]
```

• 算法特性

- **时间复杂度为 $O(n + m)$** ：非自适应排序。涉及遍历 `nums` 和遍历 `counter`，都使用线性时间。一般情况下 $n \gg m$ ，时间复杂度趋于 $O(n)$ 。
- **空间复杂度为 $O(n + m)$** ：非原地排序。借助了长度分别为 n 和 m 的数组 `res` 和 `counter`。
- **稳定排序**：由于向 `res` 中填充元素的顺序是“从右向左”，因此倒序遍历 `nums` 可以避免改变相等元素之间的相对位置，从而实现稳定排序。实际上，正序遍历 `nums` 也可以得到正确的排序结果，但结果是非稳定的。

计数排序只适用于非负整数。计数排序适用于数据量 n 大但数据范围 m 较小的情况。

9. 基数排序

算法： 核心思想与计数排序一致，也通过统计个数来实现排序。基数排序利用数字各位之间的递进关系，依次对每一位进行排序，从而得到最终的排序结果。

对于一个 d 进制的数字 x ，要求取其第 k 位 x_k ，可以使用以下计算公式：

$$x_k = \left\lfloor \frac{x}{d^{k-1}} \right\rfloor \bmod d$$

以学号数据为例，假设数字的最低位是第 1 位，最高位是第 8 位，基数排序的流程如下：

1. 初始化位数： $k = 1$
2. 对学号的第 k 位执行“计数排序”。完成后，数据会根据第 k 位从小到大排序。
3. 将 k 增加 1，然后返回步骤 2。继续迭代，直到所有位都排序完成后结束。

```

def digit(num: int, exp: int) -> int:
    """获取元素 num 的第 k 位，其中  $exp = 10^{(k-1)}$ """
    # 传入 exp 而非 k 可以避免在此重复执行昂贵的次方计算
    return (num // exp) % 10


def counting_sort_digit(nums: list[int], exp: int):
    """计数排序（根据 nums 第 k 位排序）"""
    # 十进制的位范围为 0~9，因此需要长度为 10 的桶数组
    counter = [0] * 10
    n = len(nums)
    # 统计 0~9 各数字的出现次数
    for i in range(n):
        d = digit(nums[i], exp) # 获取 nums[i] 第 k 位，记为 d
        counter[d] += 1 # 统计数字 d 的出现次数
    # 求前缀和，将“出现个数”转换为“数组索引”
    for i in range(1, 10):
        counter[i] += counter[i - 1]
    # 倒序遍历，根据桶内统计结果，将各元素填入 res
    res = [0] * n
    for i in range(n - 1, -1, -1):
        d = digit(nums[i], exp)
        j = counter[d] - 1 # 获取 d 在数组中的索引 j
        res[j] = nums[i] # 将当前元素填入索引 j
        counter[d] -= 1 # 将 d 的数量减 1
    # 使用结果覆盖原数组 nums
    for i in range(n):
        nums[i] = res[i]


def radix_sort(nums: list[int]):
    """基数排序"""
    # 获取数组的最大元素，用于判断最大位数
    m = max(nums)
    # 按照从低位到高位顺序遍历
    exp = 1
    while exp <= m:
        # 对数组元素的第 k 位执行计数排序
        # k = 1 -> exp = 1
        # k = 2 -> exp = 10
        # 即  $exp = 10^{(k-1)}$ 

```

```
counting_sort_digit(nums, exp)
exp *= 10
```

• 算法特性

- **时间复杂度为 $O(nk)$** ，非自适应排序：设数据量为 n 、数据为 d 进制、最大位数为 k ，则对某一位执行计数排序使用 $O(n + d)$ 时间。排序所有 k 位使用 $O((n + d)k)$ 时间。通常情况下， d 和 k 都相对较小，时间复杂度趋向于 $O(n)$ 。
- **空间复杂度为 $O(n + d)$** ，非原地排序：与计数排序相同，基数排序需要借助长度为 n 和 d 的数组 `res` 和 `counter`。
- **稳定排序**：当计数排序稳定时，基数排序也稳定；当计数排序不稳定时，基数排序无法保证得到正确的排序结果。

相较于计数排序，基数排序适用于数值范围较大的情况，但前提是数据必须可以表示为固定位数的格式，且位数不能过大。浮点数不适合使用基数排序。

类别	排序算法	时间复杂度 (最佳)	时间复杂度 (平均)	时间复杂度 (最差)	空间复杂度 (最差)	稳定性	就地性	自适应性	基于比较
遍历排序 $O(n^2)$	选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	非稳定	原地	非自适应	比较
遍历排序 $O(n^2)$	冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	原地	自适应	比较
遍历排序 $O(n^2)$	插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	原地	自适应	比较
分治排序 $O(n \log n)$	快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	非稳定	原地	非自适应	比较
分治排序 $O(n \log n)$	归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定	非原地	非自适应	比较
分治排序 $O(n \log n)$	堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	非稳定	原地	非自适应	比较
线性排序 $O(n)$	桶排序	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	稳定	非原地	非自适应	非比较
线性排序 $O(n)$	计数排序	$O(n + m)$	$O(n + m)$	$O(n + m)$	$O(n + m)$	稳定	非原地	非自适应	非比较
线性排序 $O(n)$	基数排序	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + b)$	稳定	非原地	非自适应	非比较

参数说明

- n ：数据量大小
- k ：桶数量（桶排序中）
- m ：数据范围（计数排序中）
- k ：最大位数（基数排序中）

- b : 进制（基数排序中）

分治

在排序算法中，快速排序、归并排序、堆排序应用了分治策略。

1. 分治实现二分查找

基于分治（递归）来实现二分查找，搜索区间 $[i, j]$ 对应的子问题记为 $f(i, j)$ 。以原问题 $f(0, n - 1)$ 为起始点，通过以下步骤进行二分查找：

1. 计算搜索区间 $[i, j]$ 的中点 m ，根据它排除一半搜索区间。
2. 递归求解规模减小一半的子问题，可能为 $f(i, m - 1)$ 或 $f(m + 1, j)$ 。
3. 循环第 1 步和第 2 步，直到找到 `target` 或区间为空时返回。

```
def dfs(nums: list[int], target: int, i: int, j: int) -> int:
    """二分查找：问题 f(i, j)"""
    # 若区间为空，代表无目标元素，则返回 -1
    if i > j:
        return -1
    # 计算中点索引 m
    m = (i + j) // 2
    if nums[m] < target:
        # 递归子问题 f(m+1, j)
        return dfs(nums, target, m + 1, j)
    elif nums[m] > target:
        # 递归子问题 f(i, m-1)
        return dfs(nums, target, i, m - 1)
    else:
        # 找到目标元素，返回其索引
        return m
```

```
def binary_search(nums: list[int], target: int) -> int:
    """二分查找"""
    n = len(nums)
    # 求解问题 f(0, n-1)
    return dfs(nums, target, 0, n - 1)
```


2. 构建二叉树

Question：给定一棵二叉树的前序遍历 `preorder` 和中序遍历 `inorder` ，请从中构建二叉树，返回二叉树的根节点。假设二叉树中没有值重复的节点。

`preorder` 和 `inorder` 都可以划分为三个部分。

- 前序遍历：[根节点|左子树|右子树] 。
- 中序遍历：[左子树|根节点|右子树] 。
- 将当前树的根节点在 `preorder` 中的索引记为 i 。
- 将当前树的根节点在 `inorder` 中的索引记为 m 。
- 将当前树在 `inorder` 中的索引区间记为 $[l, r]$ 。

子树	根节点在 <code>preorder</code> 中的索引	子树在 <code>inorder</code> 中的索引区间
当前树	i	$[l, r]$
左子树	$i + 1$	$[l, m - 1]$
右子树	$i + 1 + (m - l)$	$[m + 1, r]$

```
def dfs(
    preorder: list[int],
    inorder_map: dict[int, int],
    i: int,
    l: int,
    r: int,
) -> TreeNode | None:
    """构建二叉树：分治"""
    # 子树区间为空时终止
    if r - l < 0:
        return None
    # 初始化根节点
    root = TreeNode(preorder[i])
    # 查询 m，从而划分左右子树
    m = inorder_map[preorder[i]]
    # 子问题：构建左子树
    root.left = dfs(preorder, inorder_map, i + 1, l, m - 1)
    # 子问题：构建右子树
    root.right = dfs(preorder, inorder_map, i + 1 + m - l, m + 1, r)
    # 返回根节点
    return root

def build_tree(preorder: list[int], inorder: list[int]) -> TreeNode | None:
    """构建二叉树"""
    # 初始化哈希表，存储 inorder 元素到索引的映射
    inorder_map = {val: i for i, val in enumerate(inorder)}
    root = dfs(preorder, inorder_map, 0, 0, len(inorder) - 1)
    return root
```

- 复杂度分析(设树的节点数量为 n)
 - 初始化每一个节点（执行一个递归函数 `dfs()`）使用 $O(1)$ 时间。因此总体**时间复杂度**为 $O(n)$ 。
 - 哈希表存储 `inorder` 元素到索引的映射，空间复杂度为 $O(n)$ 。在最差情况下，即二叉树退化为链表时，递归深度达到 n ，使用 $O(n)$ 的栈帧空间。因此总体**空间复杂度**为 $O(n)$ 。

3. 汉诺塔问题

Question

给定三根柱子，记为A、B和C。起始状态下，柱子A上套着 n 个圆盘，它们从上到下按照从小到大的顺序排列。我们的任务是要把这 n 个圆盘移到柱子C上，并保持它们的原有顺序不变。在移动圆盘的过程中，需要遵守以下规则：

1. 圆盘只能从一根柱子顶部拿出，从另一根柱子顶部放入。
2. 每次只能移动一个圆盘。
3. 小圆盘必须时刻位于大圆盘之上。

算法：

将原问题 $f(n)$ 划分为两个子问题 $f(n-1)$ 和一个子问题 $f(1)$ ，并按照以下顺序解决这三个子问题。

1. 将 $n-1$ 个圆盘借助 c 从 A 移至 B 。
2. 将剩余1个圆盘从 A 直接移至 c 。
3. 将 $n-1$ 个圆盘借助 A 从 B 移至 c 。

```
def move(src: list[int], tar: list[int]):  
    """移动一个圆盘"""  
    # 从 src 顶部拿出一个圆盘  
    pan = src.pop()  
    # 将圆盘放入 tar 顶部  
    tar.append(pan)  
  
def dfs(i: int, src: list[int], buf: list[int], tar: list[int]):  
    """求解汉诺塔问题 f(i)"""  
    # 若 src 只剩下一个圆盘，则直接将其移到 tar  
    if i == 1:  
        move(src, tar)  
        return  
    # 子问题 f(i-1)：将 src 顶部 i-1 个圆盘借助 tar 移到 buf  
    dfs(i - 1, src, tar, buf)  
    # 子问题 f(1)：将 src 剩余一个圆盘移到 tar  
    move(src, tar)  
    # 子问题 f(i-1)：将 buf 顶部 i-1 个圆盘借助 src 移到 tar  
    dfs(i - 1, buf, src, tar)  
  
def solve_hanota(A: list[int], B: list[int], C: list[int]):  
    """求解汉诺塔问题"""  
    n = len(A)  
    # 将 A 顶部 n 个圆盘借助 B 移到 C  
    dfs(n, A, B, C)
```

• 复杂度分析

- 汉诺塔问题形成一棵高度为 n 的递归树，时间复杂度为 $O(2^n)$ ，空间复杂度为 $O(n)$ 。

回溯

回溯是一种“算法策略”，而递归更像是一个“工具”。

框架代码 在以下框架代码中，`state` 表示问题的当前状态，`choices` 表示当前状态下可以做出的选择：

```
def backtrack(state: State, choices: list[choice], res: list[state]):  
    """ 回溯算法框架 """  
    # 判断是否为解  
    if is_solution(state):  
        # 记录解  
        record_solution(state, res)  
        # 不再继续搜索  
        return  
    # 遍历所有选择  
    for choice in choices:  
        # 剪枝：判断选择是否合法  
        if is_valid(state, choice):  
            # 尝试：做出选择，更新状态  
            make_choice(state, choice)  
            backtrack(state, choices, res)  
            # 回退：撤销选择，恢复到之前的状态  
            undo_choice(state, choice)
```

例：在二叉树中搜索所有值为7的节点，返回根节点到这些节点的路径，并要求路径中不包含值为3的节点。

```

def is_solution(state: list[TreeNode]) -> bool:
    """判断当前状态是否为解"""
    return state and state[-1].val == 7

def record_solution(state: list[TreeNode], res: list[list[TreeNode]]):
    """记录解"""
    res.append(list(state))

def is_valid(state: list[TreeNode], choice: TreeNode) -> bool:
    """判断在当前状态下，该选择是否合法"""
    return choice is not None and choice.val != 3

def make_choice(state: list[TreeNode], choice: TreeNode):
    """更新状态"""
    state.append(choice)

def undo_choice(state: list[TreeNode], choice: TreeNode):
    """恢复状态"""
    state.pop()

def backtrack(
    state: list[TreeNode], choices: list[TreeNode], res: list[list[TreeNode]]
):
    """回溯算法：例题三"""
    # 检查是否为解
    if is_solution(state):
        # 记录解
        record_solution(state, res)
    # 遍历所有选择
    for choice in choices:
        # 剪枝：检查选择是否合法
        if is_valid(state, choice):
            # 尝试：做出选择，更新状态
            make_choice(state, choice)
            # 进行下一轮选择
            backtrack(state, [choice.left, choice.right], res)
            # 回退：撤销选择，恢复到之前的状态
            undo_choice(state, choice)

res = []
backtrack(state=[], choices=[root], res=res)

```

在找到值为 7 的节点后应该继续搜索，因此需要将记录解之后的 `return` 语句删除。

1. 全排列问题

(1) 无相等元素的情况

Question：输入一个整数数组，其中不包含重复元素，返回所有可能的排列。

```
def backtrack(
    state: list[int], choices: list[int], selected: list[bool], res: list[list[int]]
):
    """回溯算法：全排列 I"""
    # 当状态长度等于元素数量时，记录解
    if len(state) == len(choices):
        res.append(list(state))
        return
    # 遍历所有选择
    for i, choice in enumerate(choices):
        # 剪枝：不允许重复选择元素
        if not selected[i]:
            # 尝试：做出选择，更新状态
            selected[i] = True
            state.append(choice)
            # 进行下一轮选择
            backtrack(state, choices, selected, res)
            # 回退：撤销选择，恢复到之前的状态
            selected[i] = False
            state.pop()

def permutations_i(nums: list[int]) -> list[list[int]]:
    """全排列 I"""
    res = []
    backtrack(state=[], choices=nums, selected=[False] * len(nums), res=res)
    return res
```

(2) 考虑相等元素的情况

Question：输入一个整数数组，数组中可能包含重复元素，返回所有不重复的排列。

```

def backtrack(
    state: list[int], choices: list[int], selected: list[bool], res: list[list[int]]
):
    """回溯算法：全排列 II"""
    # 当状态长度等于元素数量时，记录解
    if len(state) == len(choices):
        res.append(list(state))
        return
    # 遍历所有选择
    duplicated = set[int]()
    for i, choice in enumerate(choices):
        # 剪枝：不允许重复选择元素 且 不允许重复选择相等元素
        if not selected[i] and choice not in duplicated:
            # 尝试：做出选择，更新状态
            duplicated.add(choice) # 记录选择过的元素值
            selected[i] = True
            state.append(choice)
            # 进行下一轮选择
            backtrack(state, choices, selected, res)
            # 回退：撤销选择，恢复到之前的状态
            selected[i] = False
            state.pop()

def permutations_ii(nums: list[int]) -> list[list[int]]:
    """全排列 II"""
    res = []
    backtrack(state=[], choices=nums, selected=[False] * len(nums), res=res)
    return res

```

- 假设元素两两之间互不相同，则 n 个元素共有 $n!$ 种排列；在记录结果时，需要复制长度为 n 的列表，使用 $O(n)$ 时间。因此时间复杂度为 $O(n!n)$ 。
- 最大递归深度为 n ，使用 $O(n)$ 栈帧空间。selected 使用 $O(n)$ 空间。同一时刻最多共有 n 个 duplicated，使用 $O(n^2)$ 空间。因此空间复杂度为 $O(n^2)$ 。

2. 子集和问题

(1) 给定数组无重复元素，每个元素可以被选取多次

Question

给定一个正整数数组 `nums` 和一个目标正整数 `target`，找出所有可能的组合，使得组合中的元素

和等于 `target` 。给定数组无重复元素，每个元素可以被选取多次。以列表形式返回这些组合，列表中不应包含重复组合。

算法：

给定输入数组 $[x_1, x_2, \dots, x_n]$ ，设搜索过程中的选择序列为 $[x_{i_1}, x_{i_2}, \dots, x_{i_m}]$ ，则该选择序列需要满足： $i_1 \leq i_2 \leq \dots \leq i_m$ ，不满足该条件的选择序列都会造成重复，应当剪枝。

1. 初始化变量 `start`，用于指示遍历起始点。**当做出选择 x_i 后，设定下一轮从索引 i 开始遍历。**这样做就可以让选择序列满足： $i_1 \leq i_2 \leq \dots \leq i_m$ ，从而保证子集唯一。
2. 在开启搜索前，先将数组 `nums` 排序。在遍历所有选择时，**当子集和超过 `target` 时直接结束循环**，因为后边的元素更大，其子集和一定超过 `target`。
3. 省去元素和变量 `total`，**通过在 `target` 上执行减法来统计元素和**，当 `target` 等于 0 时记录解。


```

def backtrack(
    state: list[int], target: int, choices: list[int], start: int, res: list[list[int]]
):
    """回溯算法：子集和 I"""
    # 子集和等于 target 时，记录解
    if target == 0:
        res.append(list(state))
        return
    # 遍历所有选择
    # 剪枝二：从 start 开始遍历，避免生成重复子集
    for i in range(start, len(choices)):
        # 剪枝一：若子集和超过 target，则直接结束循环
        # 这是因为数组已排序，后边元素更大，子集和一定超过 target
        if target - choices[i] < 0:
            break
        # 尝试：做出选择，更新 target, start
        state.append(choices[i])
        # 进行下一轮选择
        backtrack(state, target - choices[i], choices, i, res)
        # 回退：撤销选择，恢复到之前的状态
        state.pop()

def subset_sum_i(nums: list[int], target: int) -> list[list[int]]:
    """求解子集和 I"""
    state = [] # 状态（子集）
    nums.sort() # 对 nums 进行排序
    start = 0 # 遍历起始点
    res = [] # 结果列表（子集列表）
    backtrack(state, target, nums, start, res)
    return res

```

(2) 给定数组可能包含重复元素，每个元素只可被选择一次

Question

给定一个正整数数组 `nums` 和一个目标正整数 `target`，请找出所有可能的组合，使得组合中的元素和等于 `target`。给定数组可能包含重复元素，每个元素只可被选择一次。以列表形式返回这些组合，列表中不应包含重复组合。

算法：

需要限制相等元素在每一轮中只能被选择一次。实现方式比较巧妙：由于数组是已排序的，因此相等元素都是相邻的。这意味着在某轮选择中，若当前元素与其左边元素相等，则说明它已经被选择过，因此

直接跳过当前元素。

也可以利用变量 `start` 来满足该约束：当做出选择 x_i 后，设定下一轮从索引 $i + 1$ 开始向后遍历。这样既能去除重复子集，也能避免重复选择元素。

```
def backtrack(
    state: list[int], target: int, choices: list[int], start: int, res: list[list[int]]
):
    """回溯算法：子集和 II"""
    # 子集和等于 target 时，记录解
    if target == 0:
        res.append(list(state))
        return
    # 遍历所有选择
    # 剪枝二：从 start 开始遍历，避免生成重复子集
    # 剪枝三：从 start 开始遍历，避免重复选择同一元素
    for i in range(start, len(choices)):
        # 剪枝一：若子集和超过 target，则直接结束循环
        # 这是因为数组已排序，后边元素更大，子集和一定超过 target
        if target - choices[i] < 0:
            break
        # 剪枝四：如果该元素与左边元素相等，说明该搜索分支重复，直接跳过
        if i > start and choices[i] == choices[i - 1]:
            continue
        # 尝试：做出选择，更新 target, start
        state.append(choices[i])
        # 进行下一轮选择
        backtrack(state, target - choices[i], choices, i + 1, res)
        # 回退：撤销选择，恢复到之前的状态
        state.pop()

def subset_sum_ii(nums: list[int], target: int) -> list[list[int]]:
    """求解子集和 II"""
    state = [] # 状态（子集）
    nums.sort() # 对 nums 进行排序
    start = 0 # 遍历起始点
    res = [] # 结果列表（子集列表）
    backtrack(state, target, nums, start, res)
    return res
```

3. n 皇后问题

Question

根据国际象棋的规则，皇后可以攻击与同处一行、一列或一条斜线上的棋子。给定 n 个皇后和一个 $n \times n$ 大小的棋盘，寻找使得所有皇后之间无法相互攻击的摆放方案。

- 三个约束条件：多个皇后不能在同一行、同一列、同一条对角线上。

注意到：**棋盘每行都允许且只允许放置一个皇后。**

算法：（采取逐行放置策略）

从第一行开始，在每行放置一个皇后，直至最后一行结束。

利用一个长度为 n 的布尔型数组 `cols` 记录每一列是否有皇后。在每次决定放置前，我们通过 `cols` 将已有皇后的列进行剪枝，并在回溯中动态更新 `cols` 的状态。

设棋盘中某个格子的行列索引为 (row, col) ，**主对角线上所有格子的 $row - col$ 为恒定值，次对角线上的所有格子的 $row + col$ 是恒定值。**

```

def backtrack(
    row: int,
    n: int,
    state: list[list[str]],
    res: list[list[list[str]]],
    cols: list[bool],
    diags1: list[bool],
    diags2: list[bool],
):
    """回溯算法: n 皇后"""
    # 当放置完所有行时, 记录解
    if row == n:
        res.append([list(row) for row in state])
        return
    # 遍历所有列
    for col in range(n):
        # 计算该格子对应的主对角线和次对角线
        diag1 = row - col + n - 1
        diag2 = row + col
        # 剪枝: 不允许该格子所在列、主对角线、次对角线上存在皇后
        if not cols[col] and not diags1[diag1] and not diags2[diag2]:
            # 尝试: 将皇后放置在该格子
            state[row][col] = "Q"
            cols[col] = diags1[diag1] = diags2[diag2] = True
            # 放置下一行
            backtrack(row + 1, n, state, res, cols, diags1, diags2)
            # 回退: 将该格子恢复为空位
            state[row][col] = "#"
            cols[col] = diags1[diag1] = diags2[diag2] = False

def n_queens(n: int) -> list[list[list[str]]]:
    """求解 n 皇后"""
    # 初始化 n*n 大小的棋盘, 其中 'Q' 代表皇后, '#' 代表空位
    state = [["#" for _ in range(n)] for _ in range(n)]
    cols = [False] * n # 记录列是否有皇后
    diags1 = [False] * (2 * n - 1) # 记录主对角线上是否有皇后
    diags2 = [False] * (2 * n - 1) # 记录次对角线上是否有皇后
    res = []
    backtrack(0, n, state, res, cols, diags1, diags2)
    return res

```

- 逐行放置 n 次，考虑列约束，则从第一行到最后一行分别有 $n, n-1, \dots, 2, 1$ 个选择，使用 $O(n!)$ 时间。当记录解时，需要复制矩阵 `state` 并添加进 `res`，复制操作使用 $O(n^2)$ 时间。因此，**总体时间复杂度为： $O(n! \cdot n^2)$** 。实际上，根据对角线约束的剪枝也能够大幅缩小搜索空间，因而搜索效率往往优于以上时间复杂度。
- 数组 `state` 使用 $O(n^2)$ 空间，数组 `cols`、`diags1` 和 `diags2` 皆使用 $O(n)$ 空间。最大递归深度为 n ，使用 $O(n)$ 栈帧空间。因此，**空间复杂度为： $O(n^2)$** 。

动态规划

动态规划介绍

斐波那契数列爬楼梯问题

爬楼梯

给定一个共有 n 阶的楼梯，每步可以上 1 阶或者 2 阶，有多少种方案可以爬到楼顶？

• (1) 回溯穷举

```
def backtrack(choices: list[int], state: int, n: int, res: list[int]) -> int:
    """回溯"""
    # 当爬到第 n 阶时，方案数量加 1
    if state == n:
        res[0] += 1
    # 遍历所有选择
    for choice in choices:
        # 剪枝：不允许越过第 n 阶
        if state + choice > n:
            continue
        # 尝试：做出选择，更新状态
        backtrack(choices, state + choice, n, res)
    # 回退

def climbing_stairs_backtrack(n: int) -> int:
    """爬楼梯：回溯"""
    choices = [1, 2] # 可选择向上爬 1 阶或 2 阶
    state = 0 # 从第 0 阶开始爬
    res = [0] # 使用 res[0] 记录方案数量
    backtrack(choices, state, n, res)
    return res[0]
```

- (2) 暴力搜索

```
def dfs(i: int) -> int:
    """搜索"""
    # 已知 dp[1] 和 dp[2] , 返回之
    if i == 1 or i == 2:
        return i
    # dp[i] = dp[i-1] + dp[i-2]
    count = dfs(i - 1) + dfs(i - 2)
    return count

def climbing_stairs_dfs(n: int) -> int:
    """爬楼梯: 搜索"""
    return dfs(n)
```

- (3) 记忆化搜索

```
def dfs(i: int, mem: list[int]) -> int:
    """记忆化搜索"""
    # 已知 dp[1] 和 dp[2] , 返回之
    if i == 1 or i == 2:
        return i
    # 若存在记录 dp[i] , 则直接返回之
    if mem[i] != -1:
        return mem[i]
    # dp[i] = dp[i-1] + dp[i-2]
    count = dfs(i - 1, mem) + dfs(i - 2, mem)
    # 记录 dp[i]
    mem[i] = count
    return count

def climbing_stairs_dfs_mem(n: int) -> int:
    """爬楼梯: 记忆化搜索"""
    # mem[i] 记录爬到第 i 阶的方案总数, -1 代表无记录
    mem = [-1] * (n + 1)
    return dfs(n, mem)
```

经过记忆化处理后, 所有重叠子问题都只需计算一次, 时间复杂度优化至 $O(n)$ 。

- (4) 动态规划

```
def climbing_stairs_dp(n: int) -> int:
    """爬楼梯：动态规划"""
    if n == 1 or n == 2:
        return n
    # 初始化 dp 表，用于存储子问题的解
    dp = [0] * (n + 1)
    # 初始状态：预设最小子问题的解
    dp[1], dp[2] = 1, 2
    # 状态转移：从较小子问题逐步求解较大子问题
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

- 将数组 `dp` 称为 `dp` 表， $dp[i]$ 表示状态 i 对应子问题的解。
- 将最小子问题对应的状态（第 1 阶和第 2 阶楼梯）称为初始状态。
- 将递推公式 $dp[i] = dp[i - 1] + dp[i - 2]$ 称为状态转移方程。

```
def climbing_stairs_dp_comp(n: int) -> int:
    """爬楼梯：空间优化后的动态规划"""
    if n == 1 or n == 2:
        return n
    a, b = 1, 2
    for _ in range(3, n + 1):
        a, b = b, a + b
    return b
```

省去了数组 `dp` 占用的空间，因此空间复杂度从 $O(n)$ 降至 $O(1)$ 。

动态规划问题特性

动态规划三大特性：**重叠子问题**、**最优子结构**、**无后效性**。

1. **最优子结构**：原问题的最优解是从子问题的最优解构建得来的

爬楼梯最小代价

给定一个楼梯，你每步可以上 1 阶*或者 2 阶，每一阶楼梯上都贴有一个非负整数，表示你在该台阶所需要付出的代价。给定一个非负整数数组 `cost`，其中：`cost[i]` 表示在第 i 个台阶需要付出的代价；`cost[0]` 为地面（起始点）。计算最少需要付出多少代价才能到达顶部？

状态转移方程：

$$dp[i] = \min(dp[i - 1], dp[i - 2]) + cost[i]$$

```

def min_cost_climbing_stairs_dp(cost: list[int]) -> int:
    """爬楼梯最小代价：动态规划"""
    n = len(cost) - 1
    if n == 1 or n == 2:
        return cost[n]
    # 初始化 dp 表，用于存储子问题的解
    dp = [0] * (n + 1)
    # 初始状态：预设最小子问题的解
    dp[1], dp[2] = cost[1], cost[2]
    # 状态转移：从较小子问题逐步求解较大子问题
    for i in range(3, n + 1):
        dp[i] = min(dp[i - 1], dp[i - 2]) + cost[i]
    return dp[n]

# 空间复杂度从  $O(n)$  降至  $O(1)$ 
def min_cost_climbing_stairs_dp_comp(cost: list[int]) -> int:
    """爬楼梯最小代价：空间优化后的动态规划"""
    n = len(cost) - 1
    if n == 1 or n == 2:
        return cost[n]
    a, b = cost[1], cost[2]
    for i in range(3, n + 1):
        a, b = b, min(a, b) + cost[i]
    return b

```

2. 无后效性：给定一个确定的状态，它的未来发展只与当前状态有关，而与过去经历的所有状态无关

带约束爬楼梯（已不满足无后效性）

给定一个共有 n 阶的楼梯，你每步可以上 1 阶或者 2 阶，但不能连续两轮跳 1 阶，请问有多少种方案可以爬到楼顶？

部分问题仍然可以通过扩展状态定义，使得问题重新满足无后效性。状态 $[i, j]$ 表示处在第 i 阶并且上一轮跳了 j 阶， $j \in \{1, 2\}$ 。状态转移方程为：

$$\begin{cases} dp[i, 1] = dp[i - 1, 2] \\ dp[i, 2] = dp[i - 2, 1] + dp[i - 2, 2] \end{cases}$$

最终返回 $dp[n, 1] + dp[n, 2]$ 即可。


```
def climbing_stairs_constraint_dp(n: int) -> int:
    """带约束爬楼梯：动态规划"""
    if n == 1 or n == 2:
        return 1
    # 初始化 dp 表，用于存储子问题的解
    dp = [[0] * 3 for _ in range(n + 1)]
    # 初始状态：预设最小子问题的解
    dp[1][1], dp[1][2] = 1, 0
    dp[2][1], dp[2][2] = 0, 1
    # 状态转移：从较小子问题逐步求解较大子问题
    for i in range(3, n + 1):
        dp[i][1] = dp[i - 1][2]
        dp[i][2] = dp[i - 2][1] + dp[i - 2][2]
    return dp[n][1] + dp[n][2]
```

动态规划解题思路

Question

给定一个 $n \times m$ 的二维网格 `grid`，网格中的每个单元格包含一个非负整数，表示该单元格的代价。机器人以左上角单元格为起始点，每次只能向下或者向右移动一步，直至到达右下角单元格。请返回从左上角到右下角的最小路径和。

- **第一步：思考每轮的决策，定义状态，从而得到 dp 表**

设当前格子的行列索引为 $[i, j]$ ，则向下或向右走一步后，索引变为 $[i + 1, j]$ 或 $[i, j + 1]$ 。状态应包含行索引和列索引两个变量，记为 $[i, j]$ 。状态 $[i, j]$ 对应的子问题为：从起始点 $[0, 0]$ 走到 $[i, j]$ 的最小路径和，解记为 $dp[i, j]$ 。

- **第二步：找出最优子结构，进而推导出状态转移方程**

$$dp[i, j] = \min(dp[i - 1, j], dp[i, j - 1]) + grid[i, j]$$

- **第三步：确定边界条件和状态转移顺序**

首行 $i = 0$ 和首列 $j = 0$ 是边界条件。

1. 方法一：暴力搜索

- **递归参数：**状态 $[i, j]$ 。
- **返回值：**从 $[0, 0]$ 到 $[i, j]$ 的最小路径和 $dp[i, j]$ 。
- **终止条件：**当 $i = 0$ 且 $j = 0$ 时，返回代价 $grid[0, 0]$ 。
- **剪枝：**当 $i < 0$ 或 $j < 0$ 时索引越界，此时返回代价 $+\infty$ ，代表不可行。

```
def min_path_sum_dfs(grid: list[list[int]], i: int, j: int) -> int:
    """最小路径和：暴力搜索"""
    # 若为左上角单元格，则终止搜索
    if i == 0 and j == 0:
        return grid[0][0]
    # 若行列索引越界，则返回  $+\infty$  代价
    if i < 0 or j < 0:
        return inf
    # 计算从左上角到 (i-1, j) 和 (i, j-1) 的最小路径代价
    up = min_path_sum_dfs(grid, i - 1, j)
    left = min_path_sum_dfs(grid, i, j - 1)
    # 返回从左上角到 (i, j) 的最小路径代价
    return min(left, up) + grid[i][j]
```

设 n 和 m 分别为网格的行数和列数，每个状态都有向下和向右两种选择，从左上角走到右下角总共需要 $m + n - 2$ 步，所以**最差时间复杂度**为 $O(2^{m+n})$ 。

2. 方法二：记忆化搜索

```
def min_path_sum_dfs_mem(
    grid: list[list[int]], mem: list[list[int]], i: int, j: int
) -> int:
    """最小路径和：记忆化搜索"""
    # 若为左上角单元格，则终止搜索
    if i == 0 and j == 0:
        return grid[0][0]
    # 若行列索引越界，则返回  $+\infty$  代价
    if i < 0 or j < 0:
        return inf
    # 若已有记录，则直接返回
    if mem[i][j] != -1:
        return mem[i][j]
    # 左边和上边单元格的最小路径代价
    up = min_path_sum_dfs_mem(grid, mem, i - 1, j)
    left = min_path_sum_dfs_mem(grid, mem, i, j - 1)
    # 记录并返回左上角到 (i, j) 的最小路径代价
    mem[i][j] = min(left, up) + grid[i][j]
    return mem[i][j]
```

在引入记忆化后，所有子问题的解只需计算一次，因此**时间复杂度**等于网格尺寸 $O(nm)$ 。

3. 方法三：动态规划

```

def min_path_sum_dp(grid: list[list[int]]) -> int:
    """最小路径和：动态规划"""
    n, m = len(grid), len(grid[0])
    # 初始化 dp 表
    dp = [[0] * m for _ in range(n)]
    dp[0][0] = grid[0][0]
    # 状态转移：首行
    for j in range(1, m):
        dp[0][j] = dp[0][j - 1] + grid[0][j]
    # 状态转移：首列
    for i in range(1, n):
        dp[i][0] = dp[i - 1][0] + grid[i][0]
    # 状态转移：其余行和列
    for i in range(1, n):
        for j in range(1, m):
            dp[i][j] = min(dp[i][j - 1], dp[i - 1][j]) + grid[i][j]
    return dp[n - 1][m - 1]

```

动态规划遍历了整个网格，因此**时间复杂度**为 $O(nm)$ 。

数组 dp 大小为 $n \times m$ ，因此**空间复杂度**为 $O(nm)$ 。

4. 空间优化

可以只用一个单行数组来实现 dp 表。因为数组 dp 只能表示一行的状态，所以无法提前初始化首列状态，而是在遍历每行时更新它。

```

def min_path_sum_dp_comp(grid: list[list[int]]) -> int:
    """最小路径和：空间优化后的动态规划"""
    n, m = len(grid), len(grid[0])
    # 初始化 dp 表
    dp = [0] * m
    # 状态转移：首行
    dp[0] = grid[0][0]
    for j in range(1, m):
        dp[j] = dp[j - 1] + grid[0][j]
    # 状态转移：其余行
    for i in range(1, n):
        # 状态转移：首列
        dp[0] = dp[0] + grid[i][0]
        # 状态转移：其余列
        for j in range(1, m):
            dp[j] = min(dp[j - 1], dp[j]) + grid[i][j]
    return dp[m - 1]

```

1.0-1 背包问题

Question

给定 n 个物品，第 i 个物品的重量为 $wgt[i - 1]$ ，价值为 $val[i - 1]$ ，和一个容量为 cap 的背包。每个物品只能选择一次，问在限定背包容量下能放入物品的最大价值。

- **第一步：思考每轮的决策，定义状态，从而得到 dp 表**

对于每个物品，不放入背包时背包容量不变；放入背包时背包容量减少。因此定义状态为：**当前物品编号 i 和 背包容量 c ，记为 $[i, c]$** 。状态 $[i, c]$ 对应的子问题为：**前 i 个物品在容量为 c 的背包中的最大价值，记为 $dp[i, c]$** 。

最终待求解的是 $dp[n, cap]$ ，因此需要一个尺寸为 $(n + 1) \times (cap + 1)$ 的二维 dp 表。

- **第二步：找出最优子结构，进而推导出状态转移方程**

当对物品 i 做出决策后，剩余的是前 $i - 1$ 个物品的子问题，分为两种情况：

- i. **不放入物品 i** ：背包容量不变，状态转移到 $[i - 1, c]$ ；
- ii. **放入物品 i** ：背包容量减少 $wgt[i - 1]$ ，价值增加 $val[i - 1]$ ，状态转移到 $[i - 1, c - wgt[i - 1]]$ 。

由此得到最优子结构： $dp[i, c]$ 等于不放入物品 i 和放入物品 i 两种方案中的较大值。

状态转移方程为：

$$dp[i, c] = \max(dp[i - 1, c], dp[i - 1, c - wgt[i - 1]] + val[i - 1])$$

注意：若当前物品重量 $wgt[i - 1]$ 大于剩余容量 c ，则只能选择不放入。

- **第三步：确定边界条件和状态转移顺序**

当没有物品 ($i = 0$) 或背包容量为 0 ($c = 0$) 时，最大价值为 0。因此 **首行** $dp[0, c] = 0$ 和 **首列** $dp[i, 0] = 0$ 。

当前状态 $[i, c]$ 由上方状态 $[i - 1, c]$ 和左上方状态 $[i - 1, c - wgt[i - 1]]$ 转移而来，因此通过两层循环**正序遍历**整个 dp 表即可完成填充。

1. 方法一：暴力搜索

- **递归参数**：状态 $[i, c]$ 。
- **返回值**：子问题的解 $dp[i, c]$ 。
- **终止条件**：当物品编号越界 $i = 0$ 或背包剩余容量为 0 时，终止递归并返回价值 0。
- **剪枝**：若当前物品重量超出背包剩余容量，则只能选择不放入背包。

```
def knapsack_dfs(wgt: list[int], val: list[int], i: int, c: int) -> int:
    """0-1 背包: 暴力搜索"""
    # 若已选完所有物品或背包无剩余容量, 则返回价值 0
    if i == 0 or c == 0:
        return 0
    # 若超过背包容量, 则只能选择不放入背包
    if wgt[i - 1] > c:
        return knapsack_dfs(wgt, val, i - 1, c)
    # 计算不放入和放入物品 i 的最大价值
    no = knapsack_dfs(wgt, val, i - 1, c)
    yes = knapsack_dfs(wgt, val, i - 1, c - wgt[i - 1]) + val[i - 1]
    # 返回两种方案中价值更大的那一个
    return max(no, yes)
```

由于每个物品都会产生不选和选两条搜索分支, 因此**时间复杂度**为 $O(2^n)$ 。

2. 方法二: 记忆化搜索

借助记忆列表 `mem` 来记录子问题的解, 其中 `mem[i][c]` 对应 $dp[i, c]$ 。

```
def knapsack_dfs_mem(
    wgt: list[int], val: list[int], mem: list[list[int]], i: int, c: int
) -> int:
    """0-1 背包: 记忆化搜索"""
    # 若已选完所有物品或背包无剩余容量, 则返回价值 0
    if i == 0 or c == 0:
        return 0
    # 若已有记录, 则直接返回
    if mem[i][c] != -1:
        return mem[i][c]
    # 若超过背包容量, 则只能选择不放入背包
    if wgt[i - 1] > c:
        return knapsack_dfs_mem(wgt, val, mem, i - 1, c)
    # 计算不放入和放入物品 i 的最大价值
    no = knapsack_dfs_mem(wgt, val, mem, i - 1, c)
    yes = knapsack_dfs_mem(wgt, val, mem, i - 1, c - wgt[i - 1]) + val[i - 1]
    # 记录并返回两种方案中价值更大的那一个
    mem[i][c] = max(no, yes)
    return mem[i][c]
```

时间复杂度取决于子问题数量, 为 $O(n \times cap)$ 。

3. 方法三: 动态规划

在状态转移中填充 dp 表

```
def knapsack_dp(wgt: list[int], val: list[int], cap: int) -> int:
    """0-1 背包: 动态规划"""
    n = len(wgt)
    # 初始化 dp 表
    dp = [[0] * (cap + 1) for _ in range(n + 1)]
    # 状态转移
    for i in range(1, n + 1):
        for c in range(1, cap + 1):
            if wgt[i - 1] > c:
                # 若超过背包容量, 则不选物品 i
                dp[i][c] = dp[i - 1][c]
            else:
                # 不选和选物品 i 这两种方案的较大值
                dp[i][c] = max(dp[i - 1][c], dp[i - 1][c - wgt[i - 1]] + val[i - 1])
    return dp[n][cap]
```

时间复杂度和空间复杂度都由数组 `dp` 大小决定, 即 $O(n \times cap)$ 。

4. 空间优化

- 如果采取正序遍历, 那么遍历到 `dp[i, j]` 时, 左上方 `dp[i-1, 1] ~ dp[i-1, j-1]` 值可能已经被覆盖, 此时就无法得到正确的状态转移结果。
- 采取倒序遍历则不会发生覆盖问题, 状态转移可以正确进行。

```
def knapsack_dp_comp(wgt: list[int], val: list[int], cap: int) -> int:
    """0-1 背包: 空间优化后的动态规划"""
    n = len(wgt)
    # 初始化 dp 表
    dp = [0] * (cap + 1)
    # 状态转移
    for i in range(1, n + 1):
        # 倒序遍历
        for c in range(cap, 0, -1):
            if wgt[i - 1] > c:
                # 若超过背包容量, 则不选物品 i
                dp[c] = dp[c]
            else:
                # 不选和选物品 i 这两种方案的较大值
                dp[c] = max(dp[c], dp[c - wgt[i - 1]] + val[i - 1])
    return dp[cap]
```

空间复杂度从 $O(n^2)$ 降至 $O(n)$ 。

2. 完全背包问题

Question

给定 n 个物品，第 i 个物品的重量为 $wgt[i - 1]$ ，价值为 $val[i - 1]$ ，和一个容量为 cap 的背包。**每个物品可以重复选取**，问在限定背包容量下能放入物品的最大价值。

- 在 0-1 背包问题中，每种物品只有一个，因此将物品 i 放入背包后，只能从前 $i - 1$ 个物品中选择。
- 在完全背包问题中，每种物品的数量是无限的，因此将物品 i 放入背包后，仍可以从前 i 个物品中选择。

在完全背包问题的规则下，状态 $[i, c]$ 的变化分为两种情况：

- 不放入物品 i** ：与 0-1 背包问题相同，转移至 $[i - 1, c]$ 。
- 放入物品 i** ：与 0-1 背包问题不同，转移至 $[i, c - wgt[i - 1]]$ 。

从而状态转移方程变为：

$$dp[i, c] = \max(dp[i - 1, c], dp[i, c - wgt[i - 1]] + val[i - 1])$$

1. 动态规划

```
def unbounded_knapsack_dp(wgt: list[int], val: list[int], cap: int) -> int:
    """完全背包：动态规划"""
    n = len(wgt)
    # 初始化 dp 表
    dp = [[0] * (cap + 1) for _ in range(n + 1)]
    # 状态转移
    for i in range(1, n + 1):
        for c in range(1, cap + 1):
            if wgt[i - 1] > c:
                # 若超过背包容量，则不选物品 i
                dp[i][c] = dp[i - 1][c]
            else:
                # 不选和选物品 i 这两种方案的较大值
                dp[i][c] = max(dp[i - 1][c], dp[i][c - wgt[i - 1]] + val[i - 1])
    return dp[n][cap]
```

状态转移中有一处从 $i - 1$ 变为 i ，其余完全一致。

2. 空间优化

```
def unbounded_knapsack_dp_comp(wgt: list[int], val: list[int], cap: int) -> int:
    """完全背包：空间优化后的动态规划"""
    n = len(wgt)
    # 初始化 dp 表
    dp = [0] * (cap + 1)
    # 状态转移
    for i in range(1, n + 1):
        # 正序遍历
        for c in range(1, cap + 1):
            if wgt[i - 1] > c:
                # 若超过背包容量，则不选物品 i
                dp[c] = dp[c]
            else:
                # 不选和选物品 i 这两种方案的较大值
                dp[c] = max(dp[c], dp[c - wgt[i - 1]] + val[i - 1])
    return dp[cap]
```

由于当前状态是从左边和上边的状态转移而来的，因此空间优化后应该对 dp 表中的每一行进行**正序遍历**。这个遍历顺序与 0-1 背包正好**相反**。

3. 零钱兑换问题

Question 零钱兑换问题 I

给定 n 种硬币，第 i 种硬币的面值为 $coins[i - 1]$ ，目标金额为 amt ，**每种硬币可以重复选取**，问能够凑出目标金额的最少硬币数量。如果无法凑出目标金额，则返回 -1 。

零钱兑换可以看作完全背包问题的一种特殊情况。

- **第一步：思考每轮的决策，定义状态，从而得到 dp 表**

状态 $[i, a]$ 对应的子问题为：**前 i 种硬币能够凑出金额 a 的最少硬币数量**，记为： $dp[i, a]$ ，二维 dp 表的尺寸为： $(n + 1) \times (amt + 1)$ 。

- **第二步：找出最优子结构，进而推导出状态转移方程**

本题与完全背包问题的状态转移方程存在以下两点差异：

- 本题要求最小值，因此需将运算符 $\max()$ 更改为 $\min()$ 。
- 优化主体是硬币数量而非商品价值，因此在选中硬币时执行 $+1$ 即可。

状态转移方程：

$$dp[i, a] = \min(dp[i - 1, a], dp[i, a - coins[i - 1]] + 1)$$

- **第三步：确定边界条件和状态转移顺序**

- 当目标金额为 0 时，凑出它的最少硬币数量为 0，即首列所有： $dp[i, 0]$ 都等于 0。

- 当无硬币时，无法凑出任意 > 0 的目标金额，即是无效解。为使状态转移方程中的 $\min()$ 函数能够识别并过滤无效解，我们考虑使用： $+\infty$ 来表示它们，即令首行所有： $dp[0, a]$ 都等于： $+\infty$ 。

大多数编程语言并未提供 $+\infty$ 变量，只能使用整型 `int` 的最大值来代替。而这又会导致大数越界：状态转移方程中的 $+1$ 操作可能发生溢出。

为此，采用数字 `amt + 1` 表示无效解，因为凑出 `amt` 的硬币数量最多为 `amt`。

最后返回前，判断： $dp[n, amt]$ 是否等于： $amt + 1$ 。若是，则返回 `-1`，代表无法凑出目标金额。

1. 动态规划

```
def coin_change_dp(coins: list[int], amt: int) -> int:
    """零钱兑换：动态规划"""
    n = len(coins)
    MAX = amt + 1
    # 初始化 dp 表
    dp = [[0] * (amt + 1) for _ in range(n + 1)]
    # 状态转移：首行首列
    for a in range(1, amt + 1):
        dp[0][a] = MAX
    # 状态转移：其余行和列
    for i in range(1, n + 1):
        for a in range(1, amt + 1):
            if coins[i - 1] > a:
                # 若超过目标金额，则不选硬币 i
                dp[i][a] = dp[i - 1][a]
            else:
                # 不选和选硬币 i 这两种方案的较小值
                dp[i][a] = min(dp[i - 1][a], dp[i][a - coins[i - 1]] + 1)
    return dp[n][amt] if dp[n][amt] != MAX else -1
```

2. 空间优化

零钱兑换的空间优化的处理方式和完全背包问题一致。

```
def coin_change_dp_comp(coins: list[int], amt: int) -> int:
    """零钱兑换：空间优化后的动态规划"""
    n = len(coins)
    MAX = amt + 1
    # 初始化 dp 表
    dp = [MAX] * (amt + 1)
    dp[0] = 0
    # 状态转移
    for i in range(1, n + 1):
        # 正序遍历
        for a in range(1, amt + 1):
            if coins[i - 1] > a:
                # 若超过目标金额，则不选硬币 i
                dp[a] = dp[a]
            else:
                # 不选和选硬币 i 这两种方案的较小值
                dp[a] = min(dp[a], dp[a - coins[i - 1]] + 1)
    return dp[amt] if dp[amt] != MAX else -1
```

Question 零钱兑换问题II

给定 n 种硬币，第 i 种硬币的面值为 $coins[i - 1]$ ，目标金额为 amt ，每种硬币可以重复选取，问凑出目标金额的硬币组合数量。

本题目标是求组合数，因此子问题变为：**前 i 种硬币能够凑出金额 a 的组合数量。**

而 dp 表仍然是尺寸为 $(n + 1) \times (amt + 1)$ 的二维矩阵。当前状态的组合数量等于不选当前硬币与选择当前硬币这两种决策的组合数量之和。

状态转移方程为：

$$dp[i, a] = dp[i - 1, a] + dp[i, a - coins[i - 1]]$$

- 当目标金额为 0 时，无须选择任何硬币即可凑出目标金额，因此应将首列所有 $dp[i, 0]$ 初始化为 1。
- 当无硬币时，无法凑出任何 > 0 的目标金额，因此首行所有 $dp[0, a]$ 都等于 0。

1. 动态规划

```

def coin_change_ii_dp(coins: list[int], amt: int) -> int:
    """零钱兑换 II: 动态规划"""
    n = len(coins)
    # 初始化 dp 表
    dp = [[0] * (amt + 1) for _ in range(n + 1)]
    # 初始化首列
    for i in range(n + 1):
        dp[i][0] = 1
    # 状态转移
    for i in range(1, n + 1):
        for a in range(1, amt + 1):
            if coins[i - 1] > a:
                # 若超过目标金额, 则不选硬币 i
                dp[i][a] = dp[i - 1][a]
            else:
                # 不选和选硬币 i 这两种方案之和
                dp[i][a] = dp[i - 1][a] + dp[i][a - coins[i - 1]]
    return dp[n][amt]

```

2. 空间优化

```

def coin_change_ii_dp_comp(coins: list[int], amt: int) -> int:
    """零钱兑换 II: 空间优化后的动态规划"""
    n = len(coins)
    # 初始化 dp 表
    dp = [0] * (amt + 1)
    dp[0] = 1
    # 状态转移
    for i in range(1, n + 1):
        # 正序遍历
        for a in range(1, amt + 1):
            if coins[i - 1] > a:
                # 若超过目标金额, 则不选硬币 i
                dp[a] = dp[a]
            else:
                # 不选和选硬币 i 这两种方案之和
                dp[a] = dp[a] + dp[a - coins[i - 1]]
    return dp[amt]

```

4. 编辑距离问题

Question

输入两个字符串 s 和 t ，返回将 s 转换为 t 所需的最少编辑步数。

你可以在一个字符串中进行三种编辑操作：插入一个字符；删除一个字符；将字符替换为任意一个字符。

• 第一步：思考每轮的决策，定义状态，从而得到 dp 表

设字符串 s 和 t 的长度分别为 n 和 m ，我们先考虑两字符串尾部的字符 $s[n-1]$ 和 $t[m-1]$ 。

- 若 $s[n-1]$ 和 $t[m-1]$ 相同，我们可以跳过它们，直接考虑 $s[n-2]$ 和 $t[m-2]$ 。
- 若 $s[n-1]$ 和 $t[m-1]$ 不同，我们需要对 s 进行一次编辑（插入、删除、替换），使得两字符串尾部的字符相同，从而可以跳过它们，考虑规模更小的问题。

因此，状态为当前在 s 和 t 中考虑的第 i 和第 j 个字符，记为： $[i, j]$ ，状态 $[i, j]$ 对应的子问题：将 s 的前 i 个字符更改为 t 的前 j 个字符所需的最少编辑步数。至此，得到一个尺寸为： $(i+1) \times (j+1)$ 的二维 dp 表。

• 第二步：找出最优子结构，进而推导出状态转移方程

考虑子问题 $dp[i, j]$ ，其对应的两个字符串的尾部字符为： $s[i-1]$ 和 $t[j-1]$ ，可根据不同编辑操作分为以下三种情况。

- i. 在 $s[i-1]$ 之后添加 $t[j-1]$ ，则剩余子问题为： $dp[i, j-1]$ 。
- ii. 删除 $s[i-1]$ ，则剩余子问题为： $dp[i-1, j]$ 。
- iii. 将 $s[i-1]$ 替换为 $t[j-1]$ ，则剩余子问题为： $dp[i-1, j-1]$ 。

可得最优子结构： $dp[i, j]$ 的最少编辑步数等于 $dp[i, j-1]$ 、 $dp[i-1, j]$ 、 $dp[i-1, j-1]$ 三者中的最小编辑步数，再加上本次的编辑步数 1。对应的状态转移方程为：

$$dp[i, j] = \min(dp[i, j-1], dp[i-1, j], dp[i-1, j-1]) + 1$$

当 $s[i-1] = t[j-1]$ 时，无须编辑当前字符，这种情况下的状态转移方程为：

$$dp[i, j] = dp[i-1, j-1]$$

• 第三步：确定边界条件和状态转移顺序

当两个字符串都为空时，编辑步数为 0，即： $dp[0, 0] = 0$ 。

当 s 为空但 t 不为空时，最少编辑步数等于 t 的长度，即首行： $dp[0, j] = j$ 。

当 s 不为空但 t 为空时，最少编辑步数等于 s 的长度，即首列： $dp[i, 0] = i$ 。

观察状态转移方程，解 $dp[i, j]$ 依赖左方、上方、左上方的解，因此通过两层循环正序遍历整个 dp 表即可。

1. 动态规划

```

def edit_distance_dp(s: str, t: str) -> int:
    """编辑距离：动态规划"""
    n, m = len(s), len(t)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    # 状态转移：首行首列
    for i in range(1, n + 1):
        dp[i][0] = i
    for j in range(1, m + 1):
        dp[0][j] = j
    # 状态转移：其余行和列
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if s[i - 1] == t[j - 1]:
                # 若两字符相等，则直接跳过此两字符
                dp[i][j] = dp[i - 1][j - 1]
            else:
                # 最少编辑步数 = 插入、删除、替换这三种操作的最少编辑步数 + 1
                dp[i][j] = min(dp[i][j - 1], dp[i - 1][j], dp[i - 1][j - 1]) + 1
    return dp[n][m]

```

2. 空间优化

由于 $dp[i, j]$ 是由上方 $dp[i - 1, j]$ 、左方 $dp[i, j - 1]$ 、左上方 $dp[i - 1, j - 1]$ 转移而来的，而正序遍历会丢失左上方 $dp[i - 1, j - 1]$ ，倒序遍历无法提前构建 $dp[i, j - 1]$ ，因此两种遍历顺序都不可取。

为此，可以使用一个变量 `leftup` 来暂存左上方的解： $dp[i - 1, j - 1]$ ，从而只需考虑左方和上方的解。

```

def edit_distance_dp_comp(s: str, t: str) -> int:
    """编辑距离：空间优化后的动态规划"""
    n, m = len(s), len(t)
    dp = [0] * (m + 1)
    # 状态转移：首行
    for j in range(1, m + 1):
        dp[j] = j
    # 状态转移：其余行
    for i in range(1, n + 1):
        # 状态转移：首列
        leftup = dp[0] # 暂存 dp[i-1, j-1]
        dp[0] += 1
        # 状态转移：其余列
        for j in range(1, m + 1):
            temp = dp[j]
            if s[i - 1] == t[j - 1]:
                # 若两字符相等，则直接跳过此两字符
                dp[j] = leftup
            else:
                # 最少编辑步数 = 插入、删除、替换这三种操作的最少编辑步数 + 1
                dp[j] = min(dp[j - 1], dp[j], leftup) + 1
            leftup = temp # 更新为下一轮的 dp[i-1, j-1]
    return dp[m]

```

贪心

贪心算法介绍

- 动态规划会根据之前阶段的所有决策来考虑当前决策，并使用过去子问题的解来构建当前子问题的解。
- 贪心算法不会考虑过去的决策，而是一路向前地进行贪心选择，不断缩小问题范围，直至问题被解决。

Question

给定 n 种硬币，第 i 种硬币的面值为 $coins[i - 1]$ ，目标金额为 amt ，每种硬币可以重复选取，问能够凑出目标金额的最少硬币数量。如果无法凑出目标金额，则返回 -1 。

贪心地选择不大于且最接近它的硬币，不断循环该步骤，直至凑出目标金额为止。

```
def coin_change_greedy(coins: list[int], amt: int) -> int:
    """零钱兑换：贪心"""
    # 假设 coins 列表有序
    i = len(coins) - 1
    count = 0
    # 循环进行贪心选择，直到无剩余金额
    while amt > 0:
        # 找到小于且最接近剩余金额的硬币
        while i > 0 and coins[i] > amt:
            i -= 1
        # 选择 coins[i]
        amt -= coins[i]
        count += 1
    # 若未找到可行方案，则返回 -1
    return count if amt == 0 else -1
```

记硬币最小面值为 $\min(\text{coins})$ ，则贪心选择最多循环 $\frac{\text{amt}}{\min(\text{coins})}$ 次，时间复杂度为 $O(\frac{\text{amt}}{\min(\text{coins})})$ 。这比动态规划解法的时间复杂度 $O(n \times \text{amt})$ 小了一个数量级。

但是，贪心算法并不一定能找到最优解。因为局部最优 $\nRightarrow$ 全局最优。

贪心算法特性

- **贪心选择性质**：只有当局部最优选择始终可以导致全局最优解时，贪心算法才能保证得到最优解。
- **最优子结构**：原问题的最优解包含子问题的最优解。

1. 分数背包问题

Question

给定 n 个物品，第 i 个物品的重量为 $wgt[i - 1]$ 、价值为 $val[i - 1]$ ，和一个容量为 cap 的背包。每个物品只能选择一次，但可以选择物品的一部分，价值根据选择的重量比例计算，问在限定背包容量下背包中物品的最大价值。

注意到此问题可以对物品任意地进行切分，并按照重量比例来计算相应价值。

算法：

1. 将物品按照单位价值从高到低进行排序。
2. 遍历所有物品，每轮贪心地选择单位价值最高的物品。
3. 若剩余背包容量不足，则使用当前物品的一部分填满背包。

```
def fractional_knapsack(wgt: list[int], val: list[int], cap: int) -> int:
    """分数背包：贪心"""
    # 创建物品列表，包含两个属性：重量、价值
    items = [Item(w, v) for w, v in zip(wgt, val)]
    # 按照单位价值 item.v / item.w 从高到低进行排序
    items.sort(key=lambda item: item.v / item.w, reverse=True)
    # 循环贪心选择
    res = 0
    for item in items:
        if item.w <= cap:
            # 若剩余容量充足，则将当前物品整个装进背包
            res += item.v
            cap -= item.w
        else:
            # 若剩余容量不足，则将当前物品的一部分装进背包
            res += (item.v / item.w) * cap
            # 已无剩余容量，因此跳出循环
            break
    return res
```

内置排序算法的时间复杂度通常为 $O(\log n)$ ，空间复杂度通常为 $O(\log n)$ 或 $O(n)$ ，取决于编程语言的具体实现。

在最差情况下，需要遍历整个物品列表，因此**时间复杂度为 $O(n)$** ，其中 n 为物品数量。由于初始化了一个 `Item` 对象列表，因此**空间复杂度为 $O(n)$** 。

2. 最大容量问题

Question

输入一个数组 ht ，其中的每个元素代表一个垂直隔板的高度。数组中的任意两个隔板，以及它们之间的空间可以组成一个容器。容器的容量等于高度和宽度的乘积（面积），其中高度由较短的隔板决定，宽度是两个隔板的数组索引之差。在数组中选择两个隔板，使得组成的容器的容量最大，返回最大容量。

容器由任意两个隔板围成，因此本题的状态为两个隔板的索引，记为 $[i, j]$ 。

设容量为 $cap[i, j]$ ，则可得计算公式：

$$cap[i, j] = \min(ht[i], ht[j]) \times (j - i)$$

设数组长度为 n ，两个隔板的组合数量（状态总数）为 $C_n^2 = \frac{n(n-1)}{2}$ 个。最直接地，可以穷举所有状态，从而求得最大容量，时间复杂度为 $O(n^2)$ 。

算法：贪心策略

选取一个状态 $[i, j]$ ，其满足索引 $i < j$ 且高度 $ht[i] < ht[j]$ ，即 i 为短板、 j 为长板。若此时将长板 j 向短板 i 靠近，则容量一定变小。只有向内收缩短板 i ，才有可能使容量变大。

1. 初始状态下，指针 i 和 j 分列数组两端。
2. 计算当前状态的容量 $cap[i, j]$ ，并更新最大容量。
3. 比较板 i 和板 j 的高度，并将短板向内移动一格。
4. 循环执行第 2. 步和第 3. 步，直至 i 和 j 相遇时结束。

```
def max_capacity(ht: list[int]) -> int:
    """最大容量：贪心"""
    # 初始化 i, j, 使其分列数组两端
    i, j = 0, len(ht) - 1
    # 初始最大容量为 0
    res = 0
    # 循环贪心选择，直至两板相遇
    while i < j:
        # 更新最大容量
        cap = min(ht[i], ht[j]) * (j - i)
        res = max(res, cap)
        # 向内移动短板
        if ht[i] < ht[j]:
            i += 1
        else:
            j -= 1
    return res
```

代码循环最多 n 轮，因此时间复杂度为 $O(n)$ 。

变量 i 、 j 、 res 使用常数大小的额外空间，因此空间复杂度为 $O(1)$ 。

3. 最大切分乘积问题

Question

给定一个正整数 n ，将其切分为至少两个正整数的和，求切分后所有整数的乘积最大是多少。

将 n 切分为 m 个整数因子，其中第 i 个因子记为 n_i ，即

$$n = \sum_{i=1}^m n_i$$

目标是求得所有整数因子的最大乘积，即

$$\max \left(\prod_{i=1}^m n_i \right)$$

假设从 n 中分出一个因子 2，则它们的乘积为 $2(n-2)$ 。我们将该乘积与 n 作比较：

$$2(n-2) \geq n$$

$$n \geq 4$$

当 $n \geq 4$ 时，切分出一个 2 后乘积会变大，这说明大于等于 4 的整数都应该被切分。

算法： 贪心策略

贪心策略一： 如果切分方案中包含 ≥ 4 的因子，那么它就应该被继续切分。最终的切分方案只应出现 1、2、3 这三种因子。

在 1、2、3 这三个因子中，显然 1 最差。当 $n = 6$ 时，有 $3 \times 3 > 2 \times 2 \times 2$ 。这意味着切分出 3 比切分出 2 更优。

贪心策略二： 在切分方案中，最多只应存在两个 2。因为三个 2 总是可以替换为两个 3，从而获得更大的乘积。

1. 输入整数 n ，从其不断地切分出因子 3，直至余数为 0、1、2。
2. 当余数为 0 时，代表 n 是 3 的倍数，因此不做任何处理。
3. 当余数为 2 时，不继续划分，保留。
4. 当余数为 1 时，由于 $2 \times 2 > 1 \times 3$ ，因此应将最后一个 3 替换为 2。

利用向下整除运算得到 3 的个数 a ，用取模运算得到余数 b ： $n = 3a + b$

```

def max_product_cutting(n: int) -> int:
    """最大切分乘积：贪心"""
    # 当 n <= 3 时，必须切分出一个 1
    if n <= 3:
        return 1 * (n - 1)
    # 贪心地切分出 3，a 为 3 的个数，b 为余数
    a, b = n // 3, n % 3
    if b == 1:
        # 当余数为 1 时，将一对 1 * 3 转化为 2 * 2
        return int(math.pow(3, a - 1)) * 2 * 2
    if b == 2:
        # 当余数为 2 时，不做处理
        return int(math.pow(3, a)) * 2
    # 当余数为 0 时，不做处理
    return int(math.pow(3, a))

```

以 Python 为例。

运算符 `**` 和函数 `pow()` 的**时间复杂度**均为 $O(\log a)$ 。函数 `math.pow()` 内部调用 C 语言库的 `pow()` 函数，其执行浮点取幂，**时间复杂度**为 $O(1)$ 。

变量 a 和 b 使用常数大小的额外空间，因此**空间复杂度**为 $O(1)$ 。